Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №12
Работа с реляционными базами данных в Python с использованием SQLAlchemy и ORM

1. Цель работы

Получить практические навыки разработки Python-приложений, взаимодействующих с реляционной базой данных посредством ORM-библиотеки SQLAlchemy: изучить основные принципы ORM, научиться описывать модели базы данных в виде Python-классов, выполнять CRUD-операции, устанавливать связи между сущностями, выполнять фильтрацию, сортировку и поиск, использовать транзакции, а также организовать разделение моделей, доступа к данным и бизнес-логики.

2. Краткие теоретические сведения

ORM (Object-Relational Mapping) — технология отображения объектов объектно-ориентированной программы на таблицы реляционной базы данных. Вместо ручного формирования SQL разработчик работает с данными через объекты: session.query(User).filter(User.age > 18).all(). Таблица отображается на класс, строка — на объект, столбец — на атрибут. Основные преимущества ORM: уменьшение количества ручного SQL, работа с данными через объекты, описание структуры БД в коде, поддержка связей между сущностями и интеграция с бизнес-логикой приложения.

SQLAlchemy состоит из нескольких механизмов: Engine (соединение с базой данных), Session (рабочая сессия ORM), Model (описание сущности), Query (формирование запросов) и Relationship (связи между сущностями). Современный стиль — SQLAlchemy 2.x с DeclarativeBase, Mapped и mapped_column().

Модель описывается классом: __tablename__ задаёт имя таблицы, Mapped[int] и mapped_column(primary_key=True) — типизированные столбцы. Связи задаются через ForeignKey на стороне «многие» и relationship с back_populates на обеих сторонах. CRUD-операции выполняются через сессию:

session.add() и commit() создают запись, session.delete() удаляет, а запросы строятся при помощи select().

Транзакция — группа операций, выполняемых атомарно: несколько изменений фиксируются одним commit(), а при ошибке выполняется rollback() и ни одно изменение не сохраняется. Это гарантирует согласованность данных при операциях, затрагивающих несколько объектов.

3. Индивидуальное задание

Вариант 5 — предметная область «Ресторан». Необходимо разработать консольное Python-приложение на SQLAlchemy ORM с сущностями Customer (клиент), Dish (блюдо) и Order (заказ). Требуется реализовать создание заказа из нескольких блюд, изменение количества блюда и расчёт стоимости заказа, а также полный набор операций: создание, получение по ID, получение списка, изменение, удаление, поиск, фильтрация (блюда по категории), сортировка (по цене), получение связанных объектов (заказы клиента) и транзакционную операцию создания заказа.

4. Проектирование базы данных

База данных ресторана состоит из четырёх таблиц: customers (клиенты), dishes (блюда), orders (заказы) и order_items (позиции заказа — промежуточная таблица связи N:M). Схема приведена на рисунке 12.1.

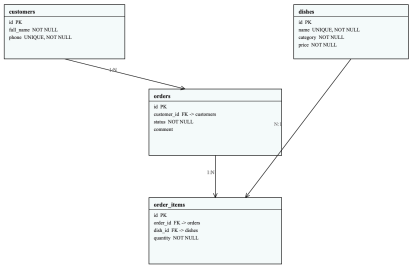


Рисунок 12.1 – ER-диаграмма базы данных ресторана

Связи: Customer 1:N Order (клиент может иметь много заказов), Order N:M Dish через промежуточную таблицу order_items с количеством позиции. У каждой таблицы есть первичный ключ id; orders содержит внешний ключ customer_id, order_items — внешние ключи order_id и dish_id. Уникальные поля: phone у customers и name у dishes. Ограничения целостности: NOT NULL для обязательных полей, CHECK реализован на уровне сервиса (положительная цена, положительное количество).

Таблица	Назначение	Ключи и ограничения
customers	клиенты ресторана	id PK, phone UNIQUE, full_name NOT NULL
dishes	блюда меню	id PK, name UNIQUE, price NOT NULL
orders	заказы клиентов	id PK, customer_id FK, status NOT NULL
order_items	позиции заказа (N:M)	id PK, order_id FK, dish_id FK, quantity NOT NULL

5. Структура проекта

```
lab_12/
├── app/
│   ├── models/entities.py        # Customer, Dish, Order, OrderItem
│   ├── repositories/repositories.py     # CustomerRepository, DishRepository
│   ├── repositories/order_repository.py # OrderRepository
│   ├── services/services.py      # сервисный слой и исключения
│   ├── database.py               # Engine, Session, создание таблиц
│   └── main.py                   # консольный интерфейс
├── tests/                        # conftest.py, test_services.py
├── requirements.txt
└── README.md
```

6. ORM-модели

Все сущности описаны в стиле SQLAlchemy 2.x: базовый класс Base(DeclarativeBase), типизированные столбцы Mapped и mapped_column, связи через ForeignKey и relationship. Связь N:M реализована с помощью модели OrderItem — она содержит внешние ключи на orders и dishes и поле quantity.

Файл: app/models/entities.py

```
from typing import List

from sqlalchemy import ForeignKey, Numeric, String, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship

from app.database import Base
```

```
class Customer(Base):
    """Гость ресторана: клиент, оформляющий заказы."""

    __tablename__ = "customers"

    id: Mapped[int] = mapped_column(primary_key=True)
    full_name: Mapped[str] = mapped_column(String(120), nullable=False)
    phone: Mapped[str] = mapped_column(String(20), nullable=False, unique=True)

    orders: Mapped[List["Order"]] = relationship(
        back_populates="customer", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"Customer(id={self.id}, name={self.full_name!r})"


class Dish(Base):
    """Блюдо из меню ресторана."""

    __tablename__ = "dishes"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(120), nullable=False, unique=True)
    category: Mapped[str] = mapped_column(String(60), nullable=False)
    price: Mapped[float] = mapped_column(Numeric(10, 2), nullable=False)

    order_items: Mapped[List["OrderItem"]] = relationship(
        back_populates="dish", cascade="all, delete-orphan"
    )

    def __repr__(self) -> str:
        return f"Dish(id={self.id}, name={self.name!r})"


class Order(Base):
    """Заказ: принадлежит клиенту и содержит несколько блюд (позиции)."""

    __tablename__ = "orders"

    id: Mapped[int] = mapped_column(primary_key=True)
    customer_id: Mapped[int] = mapped_column(ForeignKey("customers.id"),
nullable=False)
    status: Mapped[str] = mapped_column(String(20), nullable=False,
default="принят")
    comment: Mapped[str] = mapped_column(Text, nullable=True)

    customer: Mapped["Customer"] = relationship(back_populates="orders")
    items: Mapped[List["OrderItem"]] = relationship(
        back_populates="order", cascade="all, delete-orphan"
    )

    def total(self) -> float:
        return round(sum(item.subtotal() for item in self.items), 2)

    def __repr__(self) -> str:
        return f"Order(id={self.id}, status={self.status!r})"


class OrderItem(Base):
    """Позиция заказа: связь N:M между заказом и блюдом с количеством."""

    __tablename__ = "order_items"

    id: Mapped[int] = mapped_column(primary_key=True)
    order_id: Mapped[int] = mapped_column(ForeignKey("orders.id"),
nullable=False)
    dish_id: Mapped[int] = mapped_column(ForeignKey("dishes.id"),
nullable=False)
    quantity: Mapped[int] = mapped_column(nullable=False, default=1)
```

```
    order: Mapped["Order"] = relationship(back_populates="items")
    dish: Mapped["Dish"] = relationship(back_populates="order_items")

    def subtotal(self) -> float:
        return round(float(self.dish.price) * self.quantity, 2)

    def __repr__(self) -> str:
        return f"OrderItem(order={self.order_id}, dish={self.dish_id},
qty={self.quantity})"
```

Описание связей: у Customer атрибут orders: Mapped[list[Order]] = relationship(back_populates="customer", cascade="all, delete-orphan") — удаление клиента удаляет его заказы; у Order атрибуты customer и items; у OrderItem — order и dish. Такая схема соответствует связям 1:N (customers—orders) и N:M (orders—dishes через order_items).

7. Доступ к данным (Repository)

Репозитории изолируют работу с базой данных и сессией ORM. CustomerRepository и DishRepository предоставляют CRUD-операции, поиск по уникальному полю, фильтрацию по категории и сортировку; OrderRepository — операции с заказами и позициями, получение заказов клиента, поиск по статусу и транзакционное создание заказа с позициями.

Файл: app/repositories/repositories.py

```
from typing import List, Optional

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models.entities import Customer, Dish


class CustomerRepository:
    """Доступ к данным клиентов."""

    def __init__(self, session: Session):
        self._session = session

    def get(self, customer_id: int) -> Optional[Customer]:
        return self._session.get(Customer, customer_id)

    def get_all(self) -> List[Customer]:
        return
list(self._session.scalars(select(Customer).order_by(Customer.id)))

    def get_by_phone(self, phone: str) -> Optional[Customer]:
        return self._session.scalars(
            select(Customer).where(Customer.phone == phone)
        ).first()

    def create(self, full_name: str, phone: str) -> Customer:
        customer = Customer(full_name=full_name, phone=phone)
        self._session.add(customer)
        self._session.commit()
        return customer
```

```
    def update(self, customer_id: int, full_name: Optional[str] = None,
               phone: Optional[str] = None) -> Optional[Customer]:
        customer = self.get(customer_id)
        if customer is None:
            return None
        if full_name is not None:
            customer.full_name = full_name
        if phone is not None:
            customer.phone = phone
        self._session.commit()
        return customer

    def delete(self, customer_id: int) -> bool:
        customer = self.get(customer_id)
        if customer is None:
            return False
        self._session.delete(customer)
        self._session.commit()
        return True


class DishRepository:
    """Доступ к данным блюд."""

    def __init__(self, session: Session):
        self._session = session

    def get(self, dish_id: int) -> Optional[Dish]:
        return self._session.get(Dish, dish_id)

    def get_all(self) -> List[Dish]:
        return list(self._session.scalars(select(Dish).order_by(Dish.id)))

    def get_by_name(self, name: str) -> Optional[Dish]:
        return self._session.scalars(select(Dish).where(Dish.name ==
name)).first()

    # Фильтрация по категории, поиск по части названия, сортировка по цене
    def find(self, category: Optional[str] = None, search: Optional[str] = None,
             sort_by_price: bool = False) -> List[Dish]:
        stmt = select(Dish)
        if category:
            stmt = stmt.where(Dish.category == category)
        if search:
            stmt = stmt.where(Dish.name.contains(search))
        if sort_by_price:
            stmt = stmt.order_by(Dish.price)
        else:
            stmt = stmt.order_by(Dish.id)
        return list(self._session.scalars(stmt))

    def create(self, name: str, category: str, price: float) -> Dish:
        dish = Dish(name=name, category=category, price=price)
        self._session.add(dish)
        self._session.commit()
        return dish

    def update(self, dish_id: int, name: Optional[str] = None,
               category: Optional[str] = None, price: Optional[float] = None) ->
Optional[Dish]:
        dish = self.get(dish_id)
        if dish is None:
            return None
        if name is not None:
            dish.name = name
        if category is not None:
            dish.category = category
        if price is not None:
            dish.price = price
        self._session.commit()
        return dish
```

```
    def delete(self, dish_id: int) -> bool:
        dish = self.get(dish_id)
        if dish is None:
            return False
        self._session.delete(dish)
        self._session.commit()
        return True
```

Файл: app/repositories/order_repository.py

```
from typing import Dict, List, Optional, Sequence, Tuple

from sqlalchemy import select
from sqlalchemy.orm import Session

from app.models.entities import Dish, Order, OrderItem


class OrderRepository:
    """Доступ к данным заказов и их позиций."""

    def __init__(self, session: Session):
        self._session = session

    def get(self, order_id: int) -> Optional[Order]:
        return self._session.get(Order, order_id)

    def get_all(self) -> List[Order]:
        return list(self._session.scalars(select(Order).order_by(Order.id)))

    def get_by_customer(self, customer_id: int) -> List[Order]:
        return list(self._session.scalars(
            select(Order).where(Order.customer_id ==
customer_id).order_by(Order.id)
        ))

    def find(self, status: Optional[str] = None) -> List[Order]:
        stmt = select(Order)
        if status:
            stmt = stmt.where(Order.status == status)
        return list(self._session.scalars(stmt.order_by(Order.id)))

    def add_item(self, order_id: int, dish_id: int, quantity: int) -> OrderItem:
        item = OrderItem(order_id=order_id, dish_id=dish_id, quantity=quantity)
        self._session.add(item)
        self._session.commit()
        return item

    def set_item_quantity(self, item_id: int, quantity: int) ->
Optional[OrderItem]:
        item = self._session.get(OrderItem, item_id)
        if item is None:
            return None
        item.quantity = quantity
        self._session.commit()
        return item

    def add(self, order: Order) -> Order:
        self._session.add(order)
        self._session.commit()
        return order

    def update_status(self, order_id: int, status: str) -> Optional[Order]:
        order = self.get(order_id)
        if order is None:
            return None
        order.status = status
        self._session.commit()
        return order

    def delete(self, order_id: int) -> bool:
```

```
        order = self.get(order_id)
        if order is None:
            return False
        self._session.delete(order)
        self._session.commit()
        return True

    # Транзакция: проверка блюд, создание заказа, добавление позиций
    # и расчёт стоимости выполняются атомарно.
    def create_with_items(
        self, customer_id: int, items: Sequence[Tuple[int, int]], comment:
Optional[str] = None
    ) -> Order:
        order = Order(customer_id=customer_id, status="принят", comment=comment)
        self._session.add(order)
        self._session.flush()
        for dish_id, quantity in items:
            self._session.add(OrderItem(order_id=order.id, dish_id=dish_id,
quantity=quantity))
        self._session.commit()
        return order

    def stats(self) -> Dict[str, float]:
        """Агрегирующие запросы: сумма, средний чек, число заказов."""
        total = sum(order.total() for order in self.get_all())
        count = len(self.get_all())
        average = total / count if count else 0.0
        return {"count": count, "total": round(total, 2), "average":
round(average, 2)}
```

8. Бизнес-логика (Service)

Сервисный слой содержит бизнес-правила и проверки: дубликат телефона клиента или названия блюда, неположительная цена, пустой заказ, недопустимый статус, отсутствующие объекты. Ошибки представлены пользовательскими исключениями DuplicateError, NotFoundError и OrderError, поэтому приложение не завершается аварийно при некорректном вводе.

Файл: app/services/services.py

```
from typing import Dict, List, Optional, Sequence, Tuple

from app.models.entities import Customer, Dish, Order
from app.repositories.order_repository import OrderRepository
from app.repositories.repositories import CustomerRepository, DishRepository


class DuplicateError(Exception):
    """Попытка создать дубликат (одинаковый телефон клиента или название
блюда)."""


class NotFoundError(Exception):
    """Объект не найден."""


class OrderError(Exception):
    """Некорректная бизнес-операция с заказом."""


class CustomerService:
    def __init__(self, repository: CustomerRepository):
        self._repository = repository

    def list(self) -> List[Customer]:
        return self._repository.get_all()
```

```
    def get(self, customer_id: int) -> Customer:
        customer = self._repository.get(customer_id)
        if customer is None:
            raise NotFoundError(f"Клиент с ID {customer_id} не найден")
        return customer

    def create(self, full_name: str, phone: str) -> Customer:
        if self._repository.get_by_phone(phone) is not None:
            raise DuplicateError(f"Клиент с телефоном {phone} уже существует")
        if not full_name.strip():
            raise OrderError("Имя клиента не может быть пустым")
        return self._repository.create(full_name.strip(), phone.strip())

    def update(self, customer_id: int, full_name: Optional[str] = None,
               phone: Optional[str] = None) -> Customer:
        customer = self._repository.update(customer_id, full_name, phone)
        if customer is None:
            raise NotFoundError(f"Клиент с ID {customer_id} не найден")
        return customer

    def delete(self, customer_id: int) -> None:
        if not self._repository.delete(customer_id):
            raise NotFoundError(f"Клиент с ID {customer_id} не найден")


class DishService:
    def __init__(self, repository: DishRepository):
        self._repository = repository

    def list(self) -> List[Dish]:
        return self._repository.get_all()

    def get(self, dish_id: int) -> Dish:
        dish = self._repository.get(dish_id)
```

Транзакционная операция создания заказа: проверяются клиент, блюда и количество, затем в одной сессии создаётся заказ, добавляются позиции и вычисляется стоимость; при ошибке на любом этапе сессия откатывается и заказ не сохраняется. Расчёт стоимости выполняется методом Order.total() — суммой subtotal() позиций (цена блюда × количество).

```
    def set_status(self, order_id: int, status: str) -> Order:
        allowed = ("принят", "готовится", "выполнен", "отменён")
        if status not in allowed:
            raise OrderError(f"Недопустимый статус, ожидается одно из: {',
'.join(allowed)}")
        order = self._orders.update_status(order_id, status)
        if order is None:
            raise NotFoundError(f"Заказ с ID {order_id} не найден")
        return order

    def delete(self, order_id: int) -> None:
        if not self._orders.delete(order_id):
            raise NotFoundError(f"Заказ с ID {order_id} не найден")

    def total(self, order_id: int) -> float:
        return self.get(order_id).total()

    def stats(self) -> Dict[str, float]:
        return self._orders.stats()
```

9. Демонстрация работы

Тест 1. Работа консольного приложения: список клиентов, меню и список блюд (рисунок 12.2).

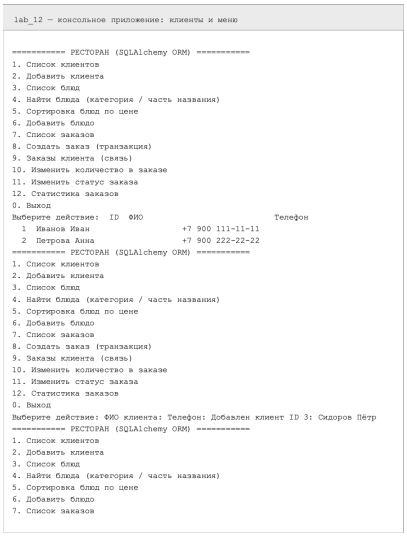
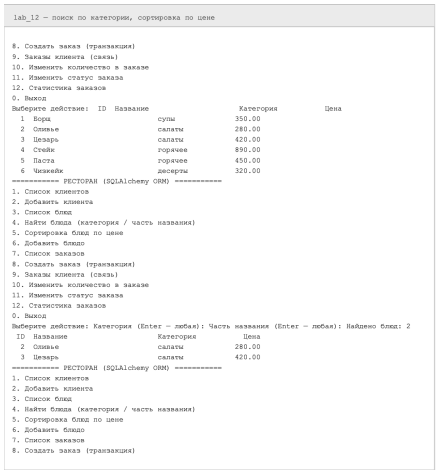


Рисунок 12.2 – Консольное приложение: клиенты и меню

Тест 2. Добавление клиента, поиск блюд по категории и части названия, сортировка по цене (рисунок 12.3).



```
lab_12 — поиск по категории, сортировка по цене

8. Создать заказ (транзакция)
9. Заказы клиента (связь)
10. Изменить количество в заказе
11. Изменить статус заказа
12. Статистика заказов
0. Выход
Выберите действие:  ID  Название                       Категория          Цена
  1  Борщ                          супы               350.00
  2  Оливье                        салаты             280.00
  3  Цезарь                        салаты             420.00
  4  Стейк                         горячее            890.00
  5  Паста                         горячее            450.00
  6  Чизкейк                       десерты            320.00
========== РЕСТОРАН (SQLAlchemy ORM) ==========
1. Список клиентов
2. Добавить клиента
3. Список блюд
4. Найти блюда (категория / часть названия)
5. Сортировка блюд по цене
6. Добавить блюдо
7. Список заказов
8. Создать заказ (транзакция)
9. Заказы клиента (связь)
10. Изменить количество в заказе
11. Изменить статус заказа
12. Статистика заказов
0. Выход
Выберите действие: Категория (Enter — любая): Часть названия (Enter — любая): Найдено блюд: 2
  ID  Название                       Категория          Цена
  2  Оливье                        салаты             280.00
  3  Цезарь                        салаты             420.00
========== РЕСТОРАН (SQLAlchemy ORM) ==========
1. Список клиентов
2. Добавить клиента
3. Список блюд
4. Найти блюда (категория / часть названия)
5. Сортировка блюд по цене
6. Добавить блюдо
7. Список заказов
8. Создать заказ (транзакция)
```

Рисунок 12.3 – Добавление клиента, поиск и сортировка

Тест 3. Добавление блюда и статистика заказов (рисунок 12.4).



Рисунок 12.4 – Добавление блюда и статистика заказов

Тест 4. Создание заказа (транзакция): заказ с блюдами, расчёт итоговой стоимости (рисунок 12.5).

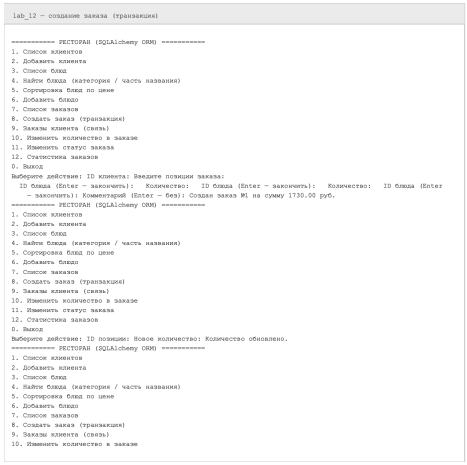


Рисунок 12.5 – Создание заказа (транзакция)

Тест 5. Изменение количества блюда, статуса заказа и получение заказов клиента через связь (рисунок 12.6).

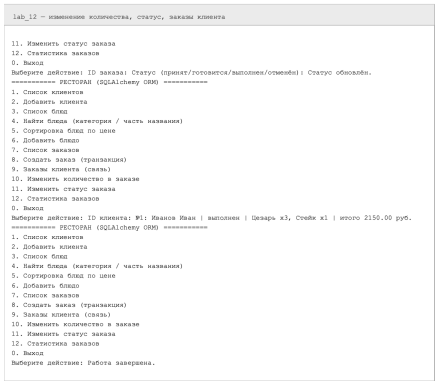


Рисунок 12.6 – Изменение количества, статуса и заказы клиента

Тест 6. Обработка ошибок: дубликат блюда, дубликат телефона клиента, несуществующий клиент, недопустимый статус (рисунок 12.7).



Рисунок 12.7 – Обработка ошибок приложения

10. Тестирование

Написано 18 автоматических тестов pytest для сервисного слоя: создание, чтение, изменение и удаление клиентов и блюд, обработка дубликатов и неположительной цены, поиск по категории и части названия, сортировка, транзакционное создание заказа, откат при несуществующем блюде, добавление позиций, изменение количества, получение заказов клиента, изменение статуса и статистика. Тесты используют изолированную SQLite-базу во временной директории. Результат выполнения pytest приведён на рисунке 12.8.

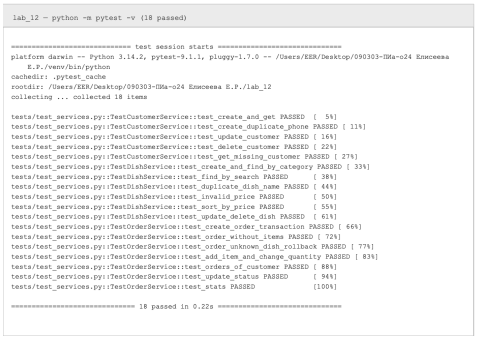


Рисунок 12.8 – Результаты выполнения pytest (18 passed)

11. Вывод

В ходе лабораторной работы разработано консольное Python-приложение «Ресторан» на SQLAlchemy ORM: спроектирована база данных из четырёх таблиц с первичными и внешними ключами, связями 1:N и N:M, уникальными ограничениями; модели описаны в стиле SQLAlchemy 2.x с типизированными столбцами; реализованы CRUD-операции, поиск, фильтрация и сортировка через сессию ORM без ручного SQL.

Приложение построено по слоям Model → Repository → Service: репозитории отвечают за доступ к данным, сервисы — за бизнес-логику и проверки. Реализована транзакционная операция создания заказа с расчётом стоимости и гарантированным откатом при ошибке, а также обработка распространённых ошибок. В результате получены практические навыки работы с реляционной базой данных через ORM, которые будут использоваться в следующих лабораторных работах при изучении FastAPI, Django и микросервисной архитектуры.